

DevOps Lab 03 – GIT and GITHUB

Lab Sec A – Friday 02:00-05:00 pm / Lab Sec B – Monday 02:00-05:00 pm

Introduction

The lab session provides a practical, hands-on introduction to Git and GitHub, two of the most widely used tools for modern software development, version control, collaboration, and DevOps workflows. The lab is designed to take students from the fundamentals of version control to working with repositories, branches, commits, remote repositories, collaboration, and automated software workflows. Students will learn how Git helps developers maintain different versions of their code, track changes, recover previous versions, work on different features, and collaborate safely without losing their work.

Few Important Definitions

1. Git

Git is a distributed version control system used to track changes in files, particularly source code, throughout the development of a software project. It allows developers to record different versions of their work using **commits**, compare changes, return to earlier versions, create separate development branches, and merge work from different developers. Because Git is distributed, each developer can have a complete local copy of the project's history and can work even without an internet connection. Git forms the underlying version-control technology used in the practical activities of these labs.

2. GitHub

GitHub is a cloud-based platform that uses Git repositories to provide online storage, collaboration, project management, and software-development features. While Git primarily manages version history on a computer, GitHub allows developers to store repositories remotely and collaborate with other people. GitHub provides features such as repositories, branches, commits, pull requests, issues, code review, project management, and GitHub Actions. In these labs, students will use GitHub as the remote platform for publishing and collaborating on their Git repositories.

3. Git Repository

A **Git repository**, commonly called a Git repo, is a project folder that Git manages for version control. It contains the project's files as well as Git's internal information about changes and history. A repository can exist locally on a student's computer and can also have a remote version hosted on platforms such as GitHub. Once a folder is initialized as a Git repository,

Git can track changes made to its files and allow those changes to be recorded through commits.

4. Local Repository

A **local repository** is the Git repository stored on a user's own computer. Developers normally make and test changes in their local repository before sharing them with others. Local Git repositories contain the project's files and its version history, allowing developers to commit changes even when they are offline. In these labs, students will initially create and work with repositories on their own computers before connecting them to GitHub.

5. Remote Repository

A **remote repository** is a copy of a Git repository stored on another computer or online platform, such as GitHub. It provides a central location where developers can share their work and obtain changes made by other team members. A local repository can be connected to one or more remote repositories. In our practical exercises, GitHub will serve as the remote repository, while the student's computer will contain the local repository.

6. Git Bash

Git Bash is a command-line environment installed with Git for Windows. It provides access to Git commands as well as many Unix/Linux-style shell commands while working on a Windows computer. Developers can use Git Bash to perform operations such as creating repositories, checking repository status, staging files, committing changes, creating branches, pulling changes, and pushing changes to GitHub. Although these labs will also demonstrate graphical interfaces, Git Bash is important because it helps students understand and use Git directly through commands.

7. GitHub Desktop

GitHub Desktop is a graphical application developed by GitHub that provides a visual interface for common Git and GitHub operations. Instead of entering commands in a terminal, users can use buttons and menus to view changes, create commits, switch branches, pull and push changes, and manage repositories. GitHub Desktop can be particularly useful for beginners who want to understand Git workflows visually. Students will be introduced to GitHub Desktop and will compare its workflow with command-line Git and the Git integration available in Visual Studio Code.

8. Visual Studio Code Git Integration

Visual Studio Code (VS Code) includes built-in Git integration that allows developers to perform many Git operations directly from the editor. Students can view changed files, stage

changes, write commit messages, create commits, switch branches, synchronize repositories, and resolve merge conflicts without leaving VS Code. In these labs, VS Code will be used extensively to demonstrate how Git can be integrated into a software development environment.

9. GitHub Repository

A **GitHub repository** is a Git repository hosted on GitHub. It can contain source code, documentation, datasets, configuration files, and other project resources. A repository can be public, allowing others to view its contents, or private, restricting access to authorized users. GitHub repositories also provide additional features such as Issues, Pull Requests, Actions, and collaboration controls.

10. Commit

A **commit** is a recorded snapshot of changes in a Git repository. Whenever a developer reaches a meaningful point in their work, they can create a commit with a message describing what was changed. Commits create a history of the project and allow developers to inspect or return to previous states. A commit is initially created in the local repository and can later be pushed to GitHub.

11. Staging

Staging is the process of selecting which changes should be included in the next Git commit. Git provides a staging area between the working files and the commit history. This allows developers to make several changes in their project but choose only specific changes for a particular commit. In VS Code, students will see a + button that can be used to stage a changed file.

12. Working Directory

The **working directory** is the current set of project files that a developer is actively editing on their computer. When a developer changes a file, Git detects that the working version differs from the most recently committed version. The developer can then review, stage, and commit those changes. Understanding the relationship between the working directory, staging area, and repository history is fundamental to understanding Git.

13. Push

Push is the Git operation used to send local commits to a remote repository such as GitHub. For example, after creating and committing a change locally, a developer can push the commit to GitHub so that the remote repository contains the new version. Push therefore

represents the movement of committed changes from the local repository to the remote repository.

14. Pull

Pull is used to obtain changes from a remote repository and integrate them into the current local branch. For example, if another developer has pushed a change to GitHub, a developer can pull those changes into their local repository. Pull generally involves fetching the remote changes and then integrating them into the local branch.

15. Fetch

Fetch retrieves information and commits from a remote repository without immediately integrating those changes into the current working branch. It allows developers to see what has changed on the remote repository before deciding whether and how to incorporate those changes locally. Fetch is therefore useful when a developer wants to inspect remote changes without immediately changing their working files.

16. Clone

Clone creates a local copy of an existing remote Git repository. For example, a student can clone a repository from GitHub to their computer and then work on the project locally. Cloning normally downloads the project's files and Git history and establishes a connection between the local repository and the original remote repository.

17. Branch

A **branch** is an independent line of development within a Git repository. Branches allow developers to work on features, experiments, bug fixes, or other changes without immediately modifying the main development line. Once the work is complete and reviewed, the branch can be merged into another branch. Branching is one of Git's most important features for collaborative software development.

18. Main Branch

The **main branch** is commonly used as the primary branch of a Git repository. It generally represents the main development or stable version of a project. GitHub repositories commonly use main as the default branch, although organizations can configure different branch names and workflows. Students will use the main branch in the initial exercises before learning how additional branches are created and merged.

19. Merge

Merge is the process of combining changes from one branch or line of development into another. For example, changes developed on a feature branch can be merged into the main branch after the work is completed. Git attempts to combine the changes automatically, but when two changes affect the same part of a file in incompatible ways, Git may require the developer to resolve a merge conflict manually.

20. Merge Conflict

A **merge conflict** occurs when Git cannot automatically determine how two different sets of changes should be combined. This commonly happens when two developers modify the same lines or closely related parts of a file. Git marks the conflicting section and asks the developer to decide what the final version should contain. Students will deliberately create and resolve a merge conflict during the labs so that they understand this important collaborative development situation.

21. Git Remote

A **Git remote** is a reference or connection to another repository, usually a repository hosted on a remote server such as GitHub. The remote is commonly given the name **origin**, which is simply a conventional name for the primary remote repository. Commands such as `git push origin main` and `git pull origin main` use this remote connection to exchange changes with GitHub.

22. GitHub Pull Request

A **Pull Request (PR)** is a GitHub feature used to propose changes from one branch into another. A developer can create a Pull Request to ask team members to review changes before they are merged into the target branch. Pull Requests support discussion, code review, automated checks, and collaboration, making them an important part of professional software development workflows.

23. GitHub Issues

GitHub Issues provide a way to track tasks, bugs, feature requests, questions, and other project-related work. Team members can create issues, assign them to people, add labels, discuss solutions, and close them when the work is completed. Issues are particularly useful for managing software projects beyond simply storing source code.

24. GitHub Actions

GitHub Actions is an automation and workflow platform built into GitHub. It allows developers to automatically execute tasks when specific events occur in a repository, such

as a code push, Pull Request, or release. For example, a GitHub Actions workflow can automatically install dependencies, run tests, check code quality, build an application, or deploy software. In these labs, students will receive an introduction to GitHub Actions and its role in **Continuous Integration and Continuous Deployment (CI/CD)**.

25. CI/CD

CI/CD stands for **Continuous Integration and Continuous Delivery/Deployment**. It refers to practices that automate parts of the software development process so that code can be frequently integrated, tested, built, and potentially deployed. GitHub Actions can be used to implement CI/CD pipelines by automatically running predefined workflows whenever developers make changes to a repository. Students will see a basic example of how a GitHub repository can automatically test or validate code.

26. .gitignore

The **.gitignore** file tells Git which files and folders should not normally be tracked in a repository. It is commonly used to exclude temporary files, generated files, operating-system files, virtual environments, compiled files, logs, and sensitive information such as local configuration files. Proper use of **.gitignore** helps keep repositories clean and prevents unnecessary or sensitive files from being committed.

27. Git History

Git history is the record of commits and other changes made to a repository over time. It allows developers to see what changed, when it changed, and which commit introduced a particular change. Git history is useful for understanding project development, investigating problems, reviewing previous work, and recovering earlier versions of files.

28. Git Graph

A **Git graph** is a visual representation of the relationships between commits and branches. It can show how different branches were created, where commits occurred, and where branches were merged. Git graphs are particularly useful when explaining branching, merging, and divergent development histories to beginners.

29. Version Control

Version control is a system for recording and managing changes to files over time. Instead of keeping multiple manually named copies such as `project_final`, `project_final2`, and `project_final_latest`, a version-control system maintains a structured history of changes. Git provides version control capabilities that allow developers to track, compare, restore, branch, merge, and collaborate on project files.

30. GitHub Collaboration

GitHub collaboration refers to the collection of features that allow multiple developers to work together on the same software project. These include repositories, branches, Pull Requests, code reviews, Issues, permissions, and automated workflows. A typical collaborative workflow allows developers to work on separate branches, submit their changes through Pull Requests, review each other's work, and merge approved changes into the main development branch.

31. Repository Fork

A **fork** is a personal copy of another user's or organization's repository under a different GitHub account. Forks are commonly used when a developer wants to experiment with or contribute to a project without having direct write access to the original repository. After making changes to a fork, the developer can propose those changes to the original project through a Pull Request.

32. Git Configuration

Git configuration contains settings that control how Git operates for a user or repository. One of the first configuration steps is normally setting the user's name and email address so that Git can associate commits with the correct developer. Git configuration can also contain preferences related to editors, branches, authentication, and other Git behavior.

33. Git Authentication

Git authentication is the process of verifying a user's identity when interacting with a remote repository. When connecting Git to GitHub, authentication allows GitHub to determine whether the user has permission to access or modify a repository. Modern GitHub workflows commonly use browser-based authentication, SSH keys, or personal access tokens rather than the old GitHub password-based method.

34. GitHub README

A **README** is a documentation file, commonly named README.md, that explains a project. It can contain information such as the project's purpose, installation instructions, usage instructions, technologies used, screenshots, examples, and contribution guidelines. A well-written README makes a GitHub repository easier for other developers, instructors, recruiters, and users to understand.

35. Markdown

Markdown is a lightweight formatting language commonly used for documentation on GitHub. Files such as README.md can use Markdown to create headings, lists, tables, links,

code blocks, images, and other formatted content. Students will encounter Markdown when creating and documenting GitHub repositories.

36. GitHub Profile

A **GitHub profile** represents a developer's public presence on GitHub. It can display repositories, contributions, projects, and other information about a user's work. For students, maintaining well-organized repositories and meaningful project documentation can make a GitHub profile useful as part of an academic or professional portfolio.

37. GitHub Actions Workflow

A **workflow** in GitHub Actions is an automated process defined by a configuration file, normally stored inside the `.github/workflows` directory of a repository. A workflow specifies when the automation should run and what tasks it should perform. For example, a workflow can run automatically whenever code is pushed to the main branch and can then execute Python tests.

38. Git Commands

Git commands are instructions used to interact with Git through a command-line interface. Examples include `git init` for initializing a repository, `git status` for viewing its current state, `git add` for staging changes, `git commit` for recording changes, `git push` for sending commits to GitHub, and `git pull` for retrieving and integrating remote changes. Although graphical tools can perform many of these operations, understanding the underlying commands helps students understand what Git is actually doing.

39. Git Status

The **git status** command provides a summary of the current state of a Git repository. It can show which files have been modified, which files are untracked, which files are staged, the current branch, and whether the local branch is ahead of or behind its remote branch. It is one of the most useful commands for beginners because it helps answer the question: **“What is happening in my repository right now?”**

40. Git Sync

Sync is a convenient term used by graphical Git tools such as VS Code to represent synchronizing the local repository with its remote repository. Depending on the current state of the repository, synchronization may involve pulling remote changes and pushing local commits. Students should understand that **Sync is a convenience feature**, while Git itself has more specific operations such as `fetch`, `pull`, and `push`.

Lab Exercise

2. Learning Objectives

By the end of this lab, students should be able to:

1. Install Git on Windows.
2. Understand the purpose of Git.
3. Understand the purpose of GitHub.
4. Use Git Bash.
5. Configure Git with a username and email.
6. Create a local project folder.
7. Initialize a local Git repository.
8. Create a GitHub repository.
9. Understand local and remote repositories.
10. Connect a local repository to GitHub.
11. Check the status of a Git repository.
12. Stage files.
13. Create commits.
14. Push local commits to GitHub.
15. Pull changes from GitHub.
16. Understand Fetch, Pull, Push and Sync.
17. Use Git through Visual Studio Code.
18. Understand GitHub commit history.
19. Understand and resolve a merge conflict.
20. Understand the purpose of GitHub Desktop.
21. Connect GitHub Desktop with a GitHub account.
22. Understand basic Git and GitHub best practices.

3. Software and Accounts Required

Before starting the lab, students should have:

- A Windows computer
- Internet connection
- A GitHub account
- Git for Windows
- Visual Studio Code
- GitHub Desktop

The main practical workflow in this lab will be:



PART A

INSTALLING AND UNDERSTANDING GIT

4. What is Git?

Git is a version control system used to track changes in files and projects. It allows developers to save different versions of their work, create commits, work with branches, collaborate with other developers, and recover previous versions of a project.

Git primarily works on the developer's computer.

Your Computer



Git



Local Repository

5. What is GitHub?

GitHub is an online platform that hosts Git repositories and provides collaboration and software-development features.

Git and GitHub are related but are not the same thing.

Git



Version control system

GitHub



Online platform for Git repositories

A simple way to remember this is:

Git manages your versions; GitHub provides an online place to store and collaborate on those versions.

6. Install Git for Windows

Open your web browser and search for:

Git for Windows

Download and install Git.

During installation, the default options can normally be kept.

Make sure Git is available from the command line.

After installation, Git Bash will be available from the Windows Start menu.

7. Open Git Bash

Open:

Start → Git Bash

A terminal window will appear.

You can test whether Git is installed by entering:

git --version

Press Enter.

You should receive a response similar to:

git version 2.x.x

The exact version number may be different.

If you see a Git version:

Git has been successfully installed.

PART B

GITHUB ACCOUNT AND REPOSITORY

8. Create a GitHub Account

Open GitHub in your web browser.

Create an account if you do not already have one.

After logging in, note your **GitHub username**.

For example:

student123

Your username will be different.

Important

Throughout this manual, whenever you see:

YOUR_GITHUB_USERNAME

replace it with your actual GitHub username.

For example:

<https://github.com/student123/git-repo>

9. Create a GitHub Repository

After logging into GitHub:

1. Open GitHub.
2. Click **New repository**.
3. Enter:

Repository name:

git-repo

4. Select **Public**.
5. Do **not** initialize it with a README for this exercise.
6. Do **not** add a .gitignore yet.
7. Do **not** add a license yet.
8. Click **Create repository**.

You should now have:

https://github.com/YOUR_GITHUB_USERNAME/git-repo

For example:

<https://github.com/student123/git-repo>

10. Understand What We Have Created

At this point we have:

YOUR COMPUTER

|



These are currently **two separate repositories**.

We will connect them later.

PART C

GIT CONFIGURATION

11. Configure Your Git Name

Open Git Bash.

Type:

```
git config --global user.name "Your Name"
```

Example:

```
git config --global user.name "Ali Ahmed"
```

Replace the example name with your own name.

Press **Enter**.

12. Configure Your Git Email

Enter:

```
git config --global user.email "your-email@example.com"
```

For example:

```
git config --global user.email "ali@gmail.com"
```

Use the email associated with your GitHub account where appropriate.

13. Verify Git Configuration

Enter:

```
git config --global user.name
```

Then:

```
git config --global user.email
```

Git should display the name and email that you configured.

Important

Git configuration does **not** create a repository.

It simply tells Git:

"This is the person making the commits."

PART D

CREATE A LOCAL PROJECT

14. Create a Project Folder

We will create our project in:

```
D:\git-repo
```

You can use another location if your instructor specifies one.

Create the folder:

```
D:\git-repo
```

15. Open Git Bash in the Project Folder

Open Git Bash.

Enter:

```
cd /d/git-repo
```

Press **Enter**.

To check where you are, enter:

```
pwd
```

You should see something similar to:

```
/d/git-repo
```

PART E

INITIALIZE THE LOCAL GIT REPOSITORY

16. Initialize Git

Enter:

```
git init
```

You should receive a message indicating that an empty Git repository has been initialized.

17. What Did git init Do?

git init created Git's internal repository information inside your project folder.

It means:

```
D:\git-repo
```

```
↓
```

```
git init
```

```
↓
```

```
Local Git Repository
```

Important

git init does **NOT** create a GitHub repository.

The GitHub repository was created separately through the GitHub website.

PART F

CREATE OUR FIRST FILE

18. Create hello.py

Open:

D:\git-repo

Create a file named:

hello.py

Enter:

print("Hello GitHub!")

Save the file.

Our project now contains:

git-repo

|
└─> hello.py

PART G

CHECK THE GIT STATUS

19. Use git status

Return to Git Bash.

Enter:

git status

You should see that hello.py is an **untracked file**.

Git is effectively saying:

"I can see this file, but you have not yet told me to include it in the next commit."

20. Understand Untracked Files

An untracked file is a file that exists inside the project folder but is not yet being tracked by Git.

Conceptually:



PART H

STAGING

21. Stage hello.py

Enter:

`git add hello.py`

Press **Enter**.

22. Check Status Again

Enter:

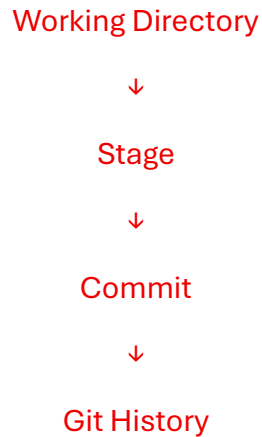
`git status`

The file should now appear as staged.

23. What is Staging?

Staging means selecting changes that should be included in the next commit.

The basic Git workflow is:



PART I

CREATE THE FIRST COMMIT

24. Create a Commit

Enter:

```
git commit -m "Initial commit"
```

You should receive information indicating that a commit was created.

25. What is a Commit?

A commit is a recorded snapshot of the project.

Our first commit represents:

Initial version of hello.py

The commit exists in the **local Git repository**.

It is not automatically on GitHub yet.

26. View Commit History

Enter:

```
git log --oneline
```

You should see your commit.

For example:

```
abc1234 Initial commit
```

The exact commit ID will be different for every student.

PART J

CONNECT LOCAL GIT TO GITHUB

27. Connect the Local Repository to GitHub

Now we connect our local repository to the GitHub repository.

Enter:

```
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/git-repo.git
```

Example

If your GitHub username is:

```
student123
```

enter:

```
git remote add origin https://github.com/student123/git-repo.git
```

Replace student123 with your own username.

28. Verify the Remote

Enter:

```
git remote -v
```

You should see something similar to:

origin `https://github.com/YOUR_GITHUB_USERNAME/git-repo.git` (fetch)

origin `https://github.com/YOUR_GITHUB_USERNAME/git-repo.git` (push)

29. What is origin?

origin is the conventional name Git gives to the main remote repository.

It is simply a short name.

Instead of repeatedly typing:

`https://github.com/YOUR_GITHUB_USERNAME/git-repo.git`

we can use:

origin

For example:

`git push origin main`

PART K

PUSH THE FIRST COMMIT TO GITHUB

30. Push the Local Repository

Enter:

`git push -u origin main`

Git may open a browser window for authentication.

Complete the GitHub authentication if requested.

31. Understand the Push

The direction is:

LOCAL COMPUTER

|

| PUSH



The commit that existed locally is now sent to GitHub.

Open your GitHub repository in the browser.

You should now see:

hello.py

32. What Does -u Mean?

The -u option establishes an upstream relationship between:

local main

and:

origin/main

After this relationship has been established, Git and graphical tools such as VS Code can more easily determine where the local branch should synchronize.

PART L

USE VISUAL STUDIO CODE

33. Open the Project in VS Code

Open Visual Studio Code.

Select:

File → Open Folder

Select:

D:\git-repo

You should see:

hello.py

in the Explorer.

34. Open the Source Control Panel

On the left side of VS Code, click the:

Source Control icon

It looks like a branching structure.

Because our repository is already connected to Git, VS Code recognizes it as a Git repository.

35. Make a Change

Open:

```
hello.py
```

Change it to:

```
print("Hello GitHub!")
```

```
name = input("What is your name? ")
```

```
print("Hello", name)
```

Save the file.

36. Observe the Source Control Panel

VS Code should show something similar to:

```
M hello.py
```

M means:

Modified

The file has changed since the last commit.

37. Stage the File in VS Code

In Source Control, click the:



next to **hello.py**.

The file moves into the staged changes area.

38. Enter a Commit Message

In the message box, enter:

Add user input to hello.py

Then click:

Commit

39. Understand What Just Happened

You have created another local commit.

The workflow was:

Edit file



Stage



Commit

The commit is currently in your local repository.

40. Push the Commit

In VS Code, you may see:

Sync Changes

or an option to push the changes.

Use:

Sync Changes

or the equivalent **Push** option.

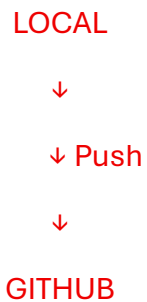
The change will be sent to GitHub.

PART M

PUSH, PULL, FETCH AND SYNC

41. Push

Push means:



Example:

`git push`

Use Push when you have local commits that you want to send to GitHub.

42. Pull

Pull means:



Pull is used when changes exist on the remote repository and you want to bring those changes into your local branch.

Example:

`git pull`

43. Fetch

Fetch retrieves information about changes from the remote repository without immediately integrating those changes into your current working branch.

Example:

`git fetch`

A simple way to remember:

Fetch = "Check/get information about remote changes."

Pull = "Get remote changes and integrate them."

44. Sync

VS Code provides **Sync Changes** as a convenient way of synchronizing local and remote changes.

Depending on the repository state, synchronization can involve:

Pull

+

Push

Therefore, Sync is a convenient graphical operation rather than a replacement for understanding Push and Pull.

45. Quick Comparison

Operation	Direction	Purpose
Push	Local → GitHub	Send local commits

Pull	GitHub → Local	Bring remote changes into local branch
Fetch	GitHub → Local Git information	Check/retrieve remote changes without integrating them
Sync	Both directions as required	Convenient synchronization

Remember:

Push sends. Pull receives and integrates. Fetch checks/retrieves. Sync coordinates synchronization.

PART N

CREATE A CHANGE DIRECTLY ON GITHUB

Now we will deliberately create a change directly on GitHub.

This will help us understand why local and remote repositories can become different.

46. Open hello.py on GitHub

Open your GitHub repository.

Click:

hello.py

Open the file.

Click the **Edit** option.

At the bottom of the file, add:

```
print("This line was added directly on GitHub.")
```

Commit the change on GitHub.

Use the commit message:

Add GitHub greeting

47. What Has Happened?

Your GitHub repository now contains a newer change.

Your local computer may not yet contain this change.

Conceptually:



The two repositories are no longer identical.

PART O

PULL THE GITHUB CHANGE

48. Pull the Changes

Return to VS Code.

Use:

Source Control → Pull

or use the terminal:

git pull

Git will retrieve the remote changes and attempt to integrate them into your local branch.

49. Understand the Pull

Before Pull:

Local ≠ GitHub

After a successful Pull:

Local = GitHub

assuming there are no additional local changes that have not been integrated.

PART P

UNDERSTANDING A MERGE CONFLICT

We will now deliberately create a situation where Git cannot automatically combine two changes.

This is an important part of understanding collaborative Git.

50. Create a Local Change

In VS Code, add at the bottom of hello.py:

```
print("This line was added locally.")
```

Commit it locally with:

Add local greeting

Do **not** synchronize yet.

51. Create Another GitHub Change

Go to GitHub.

Edit hello.py.

Add a different line in the same area:

```
print("This line was added directly on GitHub.")
```

Commit it with:

Add GitHub greeting

Now the local and remote repositories contain different changes.

52. Pull the Changes

Return to VS Code and perform:

Pull

Git may report a:

Merge Conflict

This is expected.

53. Understand the Conflict Markers

VS Code may show something like:

```
<<<<<<< HEAD
print("This line was added locally.")
=====
print("This line was added directly on GitHub.")
>>>>>>> ...
```

These markers are created by Git to show the conflicting versions.

The meaning is approximately:

```
<<<<<<< HEAD
LOCAL VERSION
=====
GITHUB VERSION
>>>>>>> ...
```

54. Resolve the Conflict

For this exercise, **keep both lines**.

Change the section to:

```
print("This line was added locally.")
print("This line was added directly on GitHub.")
```

Remove all conflict markers:

```
<<<<<<< HEAD  
=====  
>>>>>>> ...
```

Save the file.

55. Stage the Resolved File

Open the VS Code Source Control panel.

You should see the file under the merge/conflict area.

Click:

+

to stage the resolved file.

This tells Git:

"I have resolved this conflict."

56. Continue the Merge

VS Code may display:

Continue

Click:

Continue

VS Code will complete the merge process.

57. Synchronize the Repository

After the merge is completed, VS Code may display:

Sync Changes

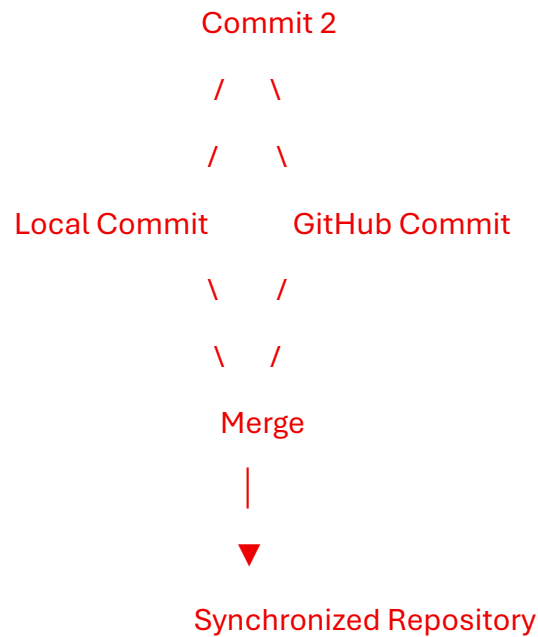
Click:

Sync Changes

After synchronization, the local and GitHub repositories should be synchronized.

58. Understand What Happened

The history conceptually looked like:



This is one of the most important concepts in collaborative version control.

PART Q

UNDERSTANDING GIT HISTORY

59. View Git History in Git Bash

Open Git Bash in your project folder.

Enter:

`git log --oneline`

You should see multiple commits.

For example:

abc1234 Merge...
def5678 Add local greeting
789abcd Add GitHub greeting
...

Your commit IDs will be different.

60. Important Concept: Commits Are Not Overwritten

When you create a new commit, Git does not simply erase the previous commit.

Instead, Git maintains a history.

Conceptually:

Commit 1
↓
Commit 2
↓
Commit 3
↓
Commit 4

This allows developers to investigate what happened to a project over time.

PART R

BASIC GIT COMMAND REFERENCE

Students should remember the following commands.

Check Git Version

`git --version`

Configure Name

`git config --global user.name "Your Name"`

Configure Email

```
git config --global user.email "your-email@example.com"
```

Initialize Repository

```
git init
```

Check Status

```
git status
```

Stage a File

```
git add hello.py
```

Stage Everything

```
git add .
```

Create Commit

```
git commit -m "Your commit message"
```

View History

```
git log --oneline
```

Add GitHub Remote

```
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/git-repo.git
```

Check Remote

```
git remote -v
```

Push

```
git push
```

Pull

```
git pull
```

Fetch

```
git fetch
```

PART S

GITHUB DESKTOP

61. What is GitHub Desktop?

GitHub Desktop is a graphical application that allows users to work with Git and GitHub without entering every Git command manually.

It provides graphical controls for operations such as:

- Viewing changes
- Creating commits
- Switching branches
- Pulling
- Pushing
- Synchronizing
- Viewing repository history
- Managing repositories

The purpose of this section is not to replace Git Bash or VS Code but to demonstrate another way of working with Git.

62. Install GitHub Desktop

Search for:

GitHub Desktop

Download and install it.

After installation, open GitHub Desktop.

63. Sign in to GitHub Desktop

When GitHub Desktop asks you to sign in:

1. Select **Sign in to GitHub.com**.
2. Your browser may open.
3. Sign in using your GitHub account.

4. Authorize GitHub Desktop if requested.
5. Return to GitHub Desktop.

Your GitHub account should now be connected.

64. Add the Existing Repository

In GitHub Desktop:

1. Open the **File** menu.
2. Select **Add Local Repository**.
3. Browse to:

D:\git-repo

4. Select the repository.
5. Click **Add Repository**.

GitHub Desktop should recognize the existing Git repository.

65. Understand the GitHub Desktop Interface

GitHub Desktop normally provides areas for:

- Current repository
- Current branch
- Changes
- History
- Commit message
- Commit button
- Push/Pull/Fetch/Sync controls

The exact appearance may vary depending on the version of GitHub Desktop.

66. Make a Change Using GitHub Desktop

Open hello.py in VS Code

Add:

```
print("Hello from GitHub Desktop!")
```

Save the file.

Return to GitHub Desktop.

The application should detect the changed file.

67. Commit Using GitHub Desktop

In GitHub Desktop:

1. Review the changed file.
2. Enter a commit message such as:

Add GitHub Desktop greeting

3. Click:

Commit to main

The change is now committed locally.

68. Push Using GitHub Desktop

After creating the local commit, GitHub Desktop may display:

Push origin

Click:

Push origin

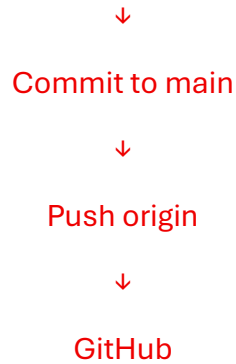
The commit will be sent to GitHub.

The basic workflow is:

Edit



Review



69. Pull Using GitHub Desktop

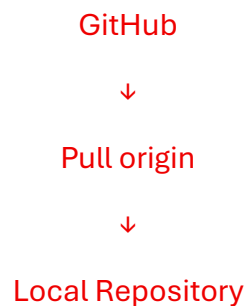
If changes have been made on GitHub by another person or through another computer, GitHub Desktop can retrieve them.

Use the available:

Pull origin

option.

The workflow becomes:



70. Sync Using GitHub Desktop

GitHub Desktop also provides synchronization-related controls.

The exact button or wording may vary by version.

The important concept remains:



Remote commits

Synchronization ensures that the local and remote repositories are brought into agreement where possible.

PART T

VS CODE VS GIT BASH VS GITHUB DESKTOP

The same Git repository can be managed using different interfaces.

Tool	Main purpose
Git Bash	Use Git commands through the terminal
VS Code Git	Manage Git while developing code
GitHub Desktop	Manage Git through a graphical application
GitHub Website	Manage repositories and collaboration online

For example, these can all ultimately perform the same type of Git operation:

- Git Bash → git commit
- VS Code → Commit button
- GitHub Desktop → Commit to main

The interface changes, but the underlying Git concepts remain the same.

PART U

USEFUL GIT AND GITHUB TIPS

71. Check Status Frequently

Use:

git status

This is one of the most useful Git commands.

It helps you understand:

- What changed?

- What is staged?
 - What is not staged?
 - Which branch am I on?
 - Are there changes that need to be pushed or pulled?
-

72. Use Meaningful Commit Messages

Avoid:

update

or:

changes

Prefer:

Add user input to hello.py

or:

Fix student login validation

A good commit message should describe what changed.

73. Commit Logical Changes

Do not wait until you have made dozens of unrelated changes.

Prefer:

Commit 1 → Add login form

Commit 2 → Add login validation

Commit 3 → Fix login error

rather than one huge commit:

Commit → Everything

74. Do Not Commit Passwords or API Keys

Never put sensitive information such as:

Passwords
API keys
tokens
private credentials

into a GitHub repository.

Even if a repository is later made private, sensitive information should not be casually committed.

75. Use .gitignore

A .gitignore file tells Git which files should not normally be tracked.

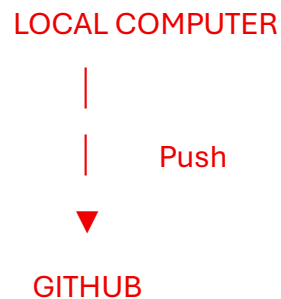
Examples include:

__pycache__/
.env
venv/
*.pyc

A .gitignore is especially important in Python and other software projects.

76. Understand the Local and Remote Difference

Always remember:





Git does not automatically upload every change you make.

PART V

COMPLETE LAB 1 WORKFLOW

Students should now understand the complete workflow used in this lab:

1. Install Git



2. Create GitHub account



3. Create GitHub repository



4. Configure Git



5. Create local project folder



6. git init



7. Create hello.py



8. git status



9. git add



10. git commit



11. git remote add origin



12. git push



13. Work in VS Code



14. Modify files



15. Stage



16. Commit



17. Push / Sync



18. Make remote change



19. Pull



20. Experience merge conflict



21. Resolve conflict



22. Stage resolved file



23. Continue merge



24. Sync



25. Explore GitHub Desktop

77. Key Concepts to Remember

Git

Version control system.

GitHub

Online platform for Git repositories and collaboration.

Repository

Project managed by Git.

Local Repository

Repository on your computer.

Remote Repository

Repository stored on a remote service such as GitHub.

Stage

Select changes for the next commit.

Commit

Record a snapshot in Git history.

Push

Send local commits to GitHub.

Pull

Bring remote changes into the local repository and integrate them.

Fetch

Retrieve remote information/changes without immediately integrating them into the current branch.

Sync

Conveniently synchronize local and remote changes.

Merge

Combine changes from different lines of development.

Merge Conflict

A situation where Git cannot automatically determine how conflicting changes should be combined.

78. Final Lab Exercise

At the end of the lab, students should have a GitHub repository containing a Python file similar to:

```
print("Hello GitHub!")

name = input("What is your name? ")
print("Hello", name)

print("This is my third commit!")

print("This line was added locally.")
print("This line was added directly on GitHub.")

print("Hello from GitHub Desktop!")
```

Students should also be able to view the **commit history** of the repository on GitHub and identify the different commits created during the laboratory.
